

Amazon 面试准备指南

Senior Product Manager – AI, NBS AI and OPS | International Sellers Group

为 Cathy Yang 定制 · Job ID 10491318 · 面试已确认: 2026-08-25 14:00-15:00 (CST) · 最新更新于 2026-08-21

说明: 这一版整合了两部分内容 —— 之前基于 JD 和简历生成的分析框架, 以及你自己准备的一份更具体、更实战的文档 (含离职原因、Why Amazon、业务理解题的完整答案、以及一个已经打磨好细节的 STAR 故事)。STAR 故事以英文为主 (面试大概率用英文, 也方便你直接照搬练习), 策略性文字用中文。带 ■ 的是需要你补充细节的地方, 带 待办的是还没做完、需要优先处理的事项。

二面准备清单

■ **面试官背景解读 Interviewer read:** Wayne Zhou 是运营背景 (Business Operation + FMCG 销售运营出身), 不是纯算法 / 技术背景。建议多用业务结果和运营效率语言 (如「降低 40% 运行时间」「效率提升 10 倍+」「MAPE 降低 5-10 个百分点」), 少陷入模型架构细节; 他大概率更关心你如何驱动跨团队落地、处理运营层面的紧急问题 (呼应第五节 Story 6 资源取舍决策、Story 7 平台稳定性), 而不是算法选型本身。可以主动用「运营」语言解释 AI 价值。

第一梯队: 现在必须搞定

1. **30-40 秒的精简版自我介绍**——一面已经讲过完整版, 二面用短版, 重点是"我能为这个团队做什么", 不是从头讲职业史。这个必须练到不用想。
2. **2 个最强 STAR 故事能脱稿讲:** Story 1 (AI 报告解读平台) 和 Story 2 (Agentic AI 0-to-1) ——已经写好了, 但"写好"不等于"能流畅讲出来", 这个要开口练, 不是看文字。
3. **Story 6 (disagree & commit, BlueStacks 资源取舍案例)**——已经补完 S/T/A/R, HM/Bar Raiser 最可能追问"influence without authority"。重点是开口练到能脱稿讲。
4. **Story 3 (主动模型监控与客户信任)**——已经补完 S/T/A/R, 是 Customer Obsession/Ownership 的强案例, 二面很可能问到。重点是开口练到能脱稿讲。

第二梯队: 理解框架即可, 不用逐字背

1. **业务理解题答案** (第五节, AI 在 NBS 的价值) ——记住"三层价值 + ML/GenAI 分工"这个结构, 不用背原文, 能现场组织语言就行。
2. **团队/岗位理解** (第一节) ——记住 NBS 是什么、JD 三大板块, 能应对"你理解这个团队在做什么"。
3. **剩余 6 个 STAR 故事** (4/5/7/8/9, 已完整) ——不用主动练, 知道"哪个故事对应哪个 LP"就行, 被问到能现挑一个讲。

○ 第四梯队: 带着文档去, 不用记

- 反问 Wayne 的问题 (第十一节, 已经帮你写好了针对 HM 的 4 个) ——面试时直接看着问都行, 不用背
- 简历薄弱点应对 (第十节) ——被问到再翻逻辑, 日常不用刻意记
- Story 10 (Jira 自动化) ——纯备胎, 没时间就不用管

建议时间分配: 如果只剩 4 天, 第一梯队至少占 2 天 (想案例+练口述), 第二梯队半天过一遍逻辑, 第三梯队今天就定掉, 第四梯队面试前一晚扫一眼就够。

一、职位与团队解读

1.1 这个团队在做什么

NBS = New Business and new Sellers（新商机与新卖家）。这个组归属 International Sellers Group，负责用 AI 提升卖家招募（seller recruitment）和账户管理（account management）的效率，解决跨境电商相关的问题。团队要求你既能定义 AI 产品策略，又能带技术团队把它做出来，还要能跟全球（CN + MP，即 marketplace/海外）的业务方对齐。

背景补充：Amazon 近期已经把面向卖家的 Seller Assistant 从生成式 AI 聊天机器人升级为 agentic AI（基于 Amazon Bedrock，使用 Nova 和 Anthropic Claude 模型），可以自主推理、规划并在卖家授权下执行动作——覆盖库存管理、账户健康监控、合规导航、广告创建、增长策略等场景。这说明 NBS AI 这个岗位很可能是这个公司级 AI-for-sellers 大战略在“新卖家”这个细分人群/生命周期阶段的落地。

1.2 JD 三大职责板块

职责板块	关键词	对应你的经历
Product Strategy & Development	AI 渗透率、卖家生命周期增长机会、seller success metrics、持续优化	DemandTec Agentic AI 0-to-1、AI 报告解读平台；BlueStacks 0-to-1 全球平台
Technical Leadership	定义技术需求与成功指标、vibe-coding、开发周期把控、数据驱动决策	Signal 副业项目；模型迭代自动化与评估框架；Jira 自动化
Stakeholder Management	跨职能协作、向高层汇报策略与影响、影响 CN 与 MP 团队对齐	DemandTec 跨团队上线筹备、主动模型监控建立客户信任；BlueStacks 用数据说服总部做资源取舍

1.3 硬性与加分资格对照

资格要求	匹配情况
7+ 年产品/项目管理经验	✓ 12+ 年，覆盖 B2B/B2C、跨境
拥有产品路线图策略经验	✓ DemandTec 三条产品线全周期策略负责人
端到端产品交付经验	✓ Agent MVP 全流程 + AI 报告解读平台 + BlueStacks 0-to-1
功能取舍（feature tradeoffs）经验	✓ Pain-point-first 单点突破策略；模型成本 vs 效果权衡
技术产品背景 / hands-on AI	✓ Signal 项目 + AI 报告解读平台架构设计
跨境电商理解（优选）	■ 相对薄弱，需要用相邻经验搭桥，见第十节
全球贸易合规知识（优选）	■ 需要诚实 + 搭桥（compliance review workstream）

二、这个岗位最该重点准备的 Leadership Principles

Leadership Principle	JD 里对应的信号	面试官想看到什么	推荐故事（点按跳转，按优先级排序）
Customer Obsession	支持卖家增长、优化卖家	你是否真的从卖家/终端用	故事 1 、 故事 3

	success metrics	户的痛点出发做决策	
Ownership	端到端产品交付、owning technology products	你是否对结果负全责，即使问题不在你直接掌控范围内	故事 2 、 故事 5 、 故事 3 、 故事 9 、 故事 7
Invent and Simplify	AI-powered solutions 、vibe-coding 简单场景开发	你是否能把复杂的 AI 能力简化成好用的 workflows	故事 1 、 故事 2 、 故事 9 、 故事 10
Think Big	global initiatives、shape global selling	你的方案是否有可扩展、可复制到更大范围的潜力	故事 5 、 故事 2
Dive Deep	定义清晰的技术需求和成功指标	你是否能讲清楚数据、模型、指标的细节	故事 3 、 故事 4 、 故事 7 、 故事 9
Are Right, A Lot	把业务机会转化成技术需求，反之亦然	你的判断是否经得起追问	故事 6 、 故事 1
Earn Trust / Have Backbone, Disagree and Commit	influence without authority 、drive alignment	你如何在没有直接汇报关系的情况下推动对齐	故事 6 、 故事 8 、 故事 1
Bias for Action / Deliver Results	fast-paced environment 、product launches	你交付的速度和确定性	故事 5 、 故事 2 、 故事 4 、 故事 8 、 故事 10
Learn and Be Curious	隐含在"技术背景"和快速学习新领域的要求里	你如何面对失败/负面反馈并成长	故事 8 、 故事 9

三、业务理解：AI 在 NBS 方向能带来什么价值

产品和竞品分析：<https://claude.ai/code/artifact/42efb389-3687-453a-9b6e-fdfbdaaed57d>

这是一个很可能被问到的开放性业务理解题，用“三层价值 + 技术选型判断”来回答，体现产品思维的层次感——这一点准备得很完整，可以直接用。

三层价值框架

层次	解决什么问题	关键动作
第一层：降低新卖家认知门槛	跨境合规复杂、选品/定价决策困难	合规材料智能预审、对话式引导、选品/定价决策辅助，知识库问答
第二层：提升内部团队人效	线下地推+客户经理体系 人力密集	线索智能分层、客户诊断助手、政策知识库问答
第三层：被动响应→主动预测	问题发现滞后、增长机会遗漏	流失预警、增长机会推送、合规风险前置提醒

关键技术判断：传统 ML 与生成式 AI 的分工（重要加分项）

不是所有场景都该用生成式 AI/LLM——这一点能体现你的技术判断力，而不只是“什么都上 AI”。

场景类型	适合的方法	原因
流失预警	传统 ML（分类/概率预测）	需要输出可比较、可标定的概率分数用于优先级排序；经过校准的模型输出是统计意义上的置信度，而 LLM 的"置信度"是语言层面表达，同样输入多次生成结果可能不一致
线索评分	传统 ML（结构化打分模型）	本质是结构化字段到分数的映射，不涉及自然语言理解；线索量级大、需要高频重新打分，传统 ML 的推理成本和延迟远低于逐条调用 LLM
选品预测	传统 ML（时序/统计预测）	核心是历史销售模式的时序预测问题，需要用真实销量校验准确率（如 MAPE）；SKU 量级通常几万到几十万，逐条调用 LLM 在成本和速度上都不现实
合规风险打分 / 异常检测（账号健康度、交易异常模式）	传统 ML（异常检测/分类）	需要高召回、可复现、可审计——监管场景要求同样输入任何时候重跑都是一致结论，这是 LLM 的随机性做不到的
合规内容判定 / 政策条文解读（listing 违规判断、沟通内容审核）	生成式 AI	本质是政策条文和具体内容的语义比对，规则本身有模糊地带，需要结合上下文判断，纯数值特征难以覆盖
非结构化文档解析与信息抽取（合同、政策文件、客户邮件）	生成式 AI	从非结构化文本里理解并抽取结构化信息，需要语言理解能力；区别于不需要 AI 的常规结构化数据 ETL 处理
对话式引导、文案生成、多语言沟通	生成式 AI	处理开放式自然语言输入，做"理解、解释、交互"

分层架构逻辑：ML 负责判断，GenAI 负责表达和交互，Agent 负责在授权边界内执行。

完整口语化回答（2-3 分钟）

我理解 NBS 的核心命题是：如何让更多新卖家更快地从注册走到规模化增长，同时不让运营成本线性增长。AI 在这里我看到三个层次的价值。第一层是降低新卖家的认知门槛.....第二层是提升内部团队的人效.....第三层，也是我认为最有价值的，是从被动响应转向主动预测.....不过我也认为这里最关键的产品判断是风险分级和技术选型：不是所有场景都该用生成式 AI。像流失预警、线索评分、选品预测这些，本质是结构化数据的预测分类问题，传统机器学习在准确性、可复现性和成本上更优；合规场景我会再拆一层——账号风险打分这类需要高召回、可复现、可审计的判断，还是传统 ML 更合适，但涉及政策条文解读、判断具体内容有没有违规的部分，因为要理解模糊地带和上下文，就需要生成式 AI 了。生成式 AI 真正的价值在于'理解、解释和交互'这一层。所以我的理解是 ML 负责判断，GenAI 负责表达和交互，Agent 负责在授权边界内执行——这也是我在 DemandTec 的真实架构经验。

延伸追问预案（业务理解，第二梯队）

1. 三层里你会先做哪一层？

Q：如果资源有限只能先做一层，你会怎么选？

我不会先入为主选一层，而是先看新卖家从招商到稳定出单这个漏斗里，哪一步的流失率或效率损耗最大。如果是最前端触达不够，那更多是第二层——给 BD 和客户经理更好的工具去覆盖更多卖家；如果是注册到稳定销售额这段漏斗里某个具体步骤流失率特别高，就要看那一步 AI 能不能介入，这时候第一层（降低卖家自己的认知门槛）和第三层（提前预测哪些卖家会卡在这一步、主动介入）其实是配合使用的，不是互斥的——提前预警的最终目的也是为了提升这个具体步骤的转化率、减少流失。以我过去做转化漏斗类问题的经验，中间的具体操作步骤（比如资质材料反复打回）往往比“卖家不知道有这个平台”更容易是大头，所以没有数据之前，我会优先怀疑是漏斗中段，但最终还是要用数据验证再决定资源怎么分。另外也会考虑竞品能力。

2. 卖家不信任 AI 建议 / 内部团队担心被 AI 替代，怎么办？

Q：如果新卖家不信任 AI 给的建议不愿意用，或者客户经理觉得 AI 在抢他们的工作，你怎么处理？

这两个问题背后其实是同一个信任问题——不管是卖家还是内部客户经理，不信任 AI 通常不是因为“AI 不够聪明”，而是因为“看不懂 AI 为什么这么判断”或者“怕被 AI 替代”。对新卖家，我坚持的原则是 AI 给建议、但最终决策权和操作权留给卖家自己，而且每一个建议都要能说清楚“为什么”，不是黑盒式地告诉他“你应该这么做”——这跟我在故事 1 里坚持的“AI 负责表达，人负责最终判断”是同一套逻辑。对内部客户经理团队，核心不是“说服”，而是“让他们先看到真实收益”：先在小范围试点，让 AI 做他们本来就觉得繁琐、不想做的重复性工作（比如批量筛选线索），而不是一上来就动他们最有成就感的部分（比如给关键客户做深度诊断建议），等他们真实体会到“这东西帮我省了时间”，接受度会自然上升——这跟我在故事 2 里“先做 demo 给客户看真实反馈，而不是内部拍脑袋定方案”是同一个方法论。

3. 跟 Amazon 现有的 Seller Assistant 是什么关系？

Q：Seller Assistant 已经是 agentic AI、覆盖合规导航/库存/广告这些场景了，NBS AI 这个岗位跟它是什么关系，是不是重复建设？

我理解 Seller Assistant 更像是一个通用能力层——面向所有卖家、覆盖全生命周期的 agentic 能力。而 NBS 这个方向，从 JD 看，聚焦的是“新卖家”这个特定人群，特别是从注册到规模化增长这个早期生命周期阶段。我理解这两者的关系更可能是分层复用，而不是重复建设：底层的能力（推理、规划、执行）大概率是共享的基础设施，但 NBS AI 要解决的是这个特定人群、这个特定阶段的问题——新卖家更需要的是“手把手的引导”和“信心建立”，跟一个已经稳定经营多年的老卖家需求完全不同，所以这个岗位的价值更多在于针对这个细分市场做深度定制和场景化设计。当然这只是我基于公开信息的判断，如果能进一步了解实际情况，我很乐意——这也是我会主动向面试官请教的一个问题。

4. 怎么衡量这个产品成功不成功？

Q：上线后你会看哪些指标？如果效率指标提升了但 GMV/留存没变化，算成功吗？

我会分两层指标看。第一层是过程指标，比如 AI 处理的任务量、响应时间、人效提升比例，这些能说明“系统在跑、在起作用”，是团队自己能快速看到的先行指标。第二层，也是更关键的一层，是结果指标——新卖家的转化率（注册到首单、到稳定出单）、留存率、以及 GMV 增长。如果只有过程指标好看、结果指标没变化，我不会认为这是成功——那说明我们可能优化的是一个不影响卖家实际结果的环节，或者优化力度还不够抵消掉别的瓶颈。我会把这种情况当成一个诊断信号，去看漏斗里还有哪个环节没被解决，而不是把过程指标本身当成 KPI 去汇报。

5. 生成式 AI 给错合规建议，责任算谁的？

Q：生成式 AI 做政策解读出现幻觉、给了错误的合规建议，责任在谁？

核心是要把“AI 输出”和“最终决策”分开看。我的原则是：凡是可能直接影响卖家合规决策、有实际业务后果的内容，AI 只能做“辅助解读和起草”，不能做“最终判定”——有明确规则的地方用确定性逻辑硬编码，不靠 LLM 推理；没有明确规则、触及“账号处罚风险”这类高后果场景，必须保留人工复核环节。具体机制上，我会对政策解

读类内容做置信度分层，低风险常见问题可以 AI 直接给建议，高风险场景必须标注"建议，请二次确认"；同时建立反馈闭环，一旦发现 AI 给错建议，能快速定位是数据问题还是模型理解问题，并迭代到测试集里防止再犯——这套机制就是我在故事 1 里搭建的幻觉控制体系，责任边界要从架构设计阶段就定清楚，而不是出问题后再补。

6. 中国卖家和海外卖家的需求有什么本质不同？

Q: CN 和 marketplace（海外）卖家在用这个 AI 产品上，需求有什么不同？

核心差异不在"需不需要 AI"，而在几个具体维度：语言和沟通习惯上，中国卖家更习惯即时、对话式的引导，海外卖家可能更适应邮件/工单这种异步沟通；合规复杂度也不对称，中国卖家做跨境面对的是"走出去"的合规复杂度（目的国税务、进口合规），海外本土卖家面对的更多是平台规则本身；信任建立的路径也不同，中国卖家对"国内团队、能说中文、快速响应"本身就有信任加成，海外卖家可能更看重标准化流程和 SLA 承诺。这些差异意味着同一套 AI 能力在 CN 和海外两端可能需要不同的交互设计和内容策略，但底层的判断逻辑（合规风险识别、流失预警）应该是可以复用的框架，只是训练数据和话术要分开打磨——这也是为什么这个岗位需要同时跟 CN 和 MP 团队对齐，不是做两套产品，而是一套底层能力、两套场景化表达。

四、工作背景

4.1 自我介绍

我是 Cathy，计算机背景，12 年跨境产品经验。之前在 DemandTec 担任产品总监，负责 Data Science、Platform 和 Data Engineering 三条产品线。

过去两年最核心的工作，是从 0 到 1 把 GenAI 能力落地到我们的定价促销决策平台，比如复杂报告的自动解读和归因分析、定价决策的可解释性生成，——拿到了多个标杆客户的认可，也进了公司今年的商业化路线图。同时在传统 ML 模型上，我们把核心模型的运行效率提升了 40%，准确率提升了 6%。

这个岗位吸引我的地方是，NBS 要解决的问题和我现在做的事高度一致：用 AI 帮业务方决策更快更准，同时控制运营成本。底层的核心问题都是——GenAI 如何更好地理解现有系统的业务逻辑，再通过 AI 增强或 Agent 自主决策的方式实现业务目标。

1. 完整工作经历

我的职业路径大概可以分成四个阶段。

我是计算机专业出身，就读于挪威奥斯陆大学，第一份工作是在 Capgemini 做 Java 后端工程师，负责银行后端系统开发，Java 工程师认证。那几年让我第一次看懂一个复杂系统内部是怎么运转的，也让我意识到，除了想把功能实现出来，我也很想参与“做什么、为什么这么做”这类判断。

第二阶段是游戏行业。我先在 Funcom 做项目经理和美术总监，参与 The Secret World、Age of Conan 这些知名项目，管三十多个美术师，负责跨国团队之间的协调对齐；之后在 6waves 做游戏制作人，独立带一款产品从概念做到上线，还尝试了不同地区的发行。这个阶段我积累了如何独立交付一款完整产品，以及怎么让一款成功的产品进入新市场这样的方法论。

第三阶段是平台型产品。我在阿里做过游戏的引入评估、开发者数据分析平台、HTML5 游戏平台，学习了阿里的平台思维以及快速集中资源试错的业务拓展逻辑；之后在 BlueStacks 待了八年，从零搭建中国的产品和工程团队，中国区产品成功之后升值为产品总监，接管全球产品线，峰值做到千万级月活。这个阶段我从“做一个产品”转向“做让内容流转起来的平台型产品”，也让我更为深刻的理解并实践了平台型产品的产品矩阵搭建以及规模化复制单一地区的成功到全球其他国家的运营拓展思路。同时，技术方面，我开始接触机器学习和算法做智能个性化推荐。

第四阶段是在 DemandTec 做以数据科学为内核的企业级 SaaS 产品。我 2023 年底加入，正好赶上生成式 AI 在企业场景落地的窗口，负责 Data Science，Data Engineering、和 Platform 三条核心产品线，一年多的时间从技术产品经理晋升到产品总监。过去两年最核心的工作，是从 0 到 1 把 GenAI 能力落地到我们的定价促销决策平台，比如复杂报告的自动解读和归因分析、定价决策的可解释性生成，——拿到了多个标杆客户的认可，也进了公司今年的商业化路线图。同时在传统 ML 模型上，我们把核心模型的运行效率提升了 40%，准确率提升了 6%。

这个岗位吸引我的地方是，NBS 要解决的问题和我现在做的事高度一致：用 AI 帮业务方决策更快更准，同时控制运营成本。底层的核心能力都是——GenAI 如何更好地理解现有系统的业务逻辑，再通过 AI 增强或 Agent 自主决策的方式实现业务目标。

如果用一句话概括，我这四个阶段其实是能力和产品认知的叠加，技术背景让我懂系统怎么跑，游戏阶段让我懂怎么端到端交付，平台阶段让我懂怎么设计产品矩阵，盈利模式，规模化的复制成功，企业级 SAAS 阶段是对企业级客户需求的精准理解和挖掘，用多种 AI 技术满足甚至超预期的实现客户需求。

一、研发工程师阶段（约 60 秒）

（被问“讲讲你最早的工程经历”或“为什么从工程转产品”时用）

我是奥斯陆大学计算机科学背景，毕业后在 Capgemini 做 Java 后端工程师，参与银行系统的开发。

银行系统是典型的强监管、高复杂度系统，模块之间的耦合关系很重，而且每一个模块背后都对应着一个真实的业务部门。我在做开发的过程中，慢慢从“我负责的这个接口”跳出来，看到了整个系统里各个模块、各个部门是怎么协同工作的。

也是在这个过程中，我第一次很明确地意识到，我对“这个功能为什么这么设计、它对哪个部门产生什么价值”的兴趣，其实超过了对“怎么把它实现出来”的兴趣。

当时我判断了一件事，这个判断影响了我后面所有的选择：一个有工程背景的人去做产品，最大的优势不在于会写代码，而在于**能判断一个需求的实现难度和技术权衡到底在哪里**，不会提出不切实际的方案，也不会被“这个做不了”轻易挡回来。所以我选择了往产品方向走。

这条线一直到今天都是连续的——我现在还会自己用 Web Coding 搭原型、接真实业务数据做 Demo，也能跟数据科学家直接讨论模型评估的逻辑，这个底子那个阶段打下来的。

二、游戏制作阶段

2.1 Funcom：跨国团队的需求对齐（约 75 秒）

（被问“跨文化协作经历”或“怎么处理团队之间的理解偏差”时用）

在 Funcom 我是项目经理兼美术总监，参与 The Secret World、Age of Conan 这些项目，管理三十多名美术师，团队分布在奥斯陆和中国两地。

我举一个当时很典型的场景。奥斯陆的动画总监对一个 monster 的动画有非常具体的要求，但中国这边的动画师理解不到位，做出来的东西反复达不到预期，来回返工了好几轮。

我做了两件事。第一件是我直接介入到双方的讨论里，不只是做语言上的翻译，而是帮双方对齐理解——总监想要的效果到底是什么、中国团队觉得难在哪里，然后在“需求标准”和“实现难度”之间找一个双方都能接受的折中方案。

第二件事，也是更关键的一件，是我在团队把作品提交给总监做正式 review 之前，**先按我自己对需求的理解做一轮预审**。因为我全程参与了需求讨论，我大致知道总监会在哪些点上卡住，所以我可以提前把偏差找出来、让团队先改一轮。

结果是返工次数明显下降，整体交付效率提升了。而且这个机制建立起来之后，总监对我的判断产生了信任，后续类似的对齐工作更多地直接交给我来处理。

这段经历让我确认了一件事：我在项目管理和跨文化协同上是游刃有余的，而且我更擅长的不是事后救火，而是**提前设计一个机制把问题挡在前面**。

2.2 6waves：用数据把产品推进新市场（约 75 秒）

（被问“独立负责产品的经历”或“数据驱动决策”时用，这是一个强案例）

在 6waves 我是游戏制作人，独立带一款解谜类游戏，从概念、开发一直到上线和海外发行，全流程我一个人负责。

产品先在 Facebook 上线，拿到了 4.3 分以上的用户评分，表现是不错的。但真正我觉得有价值的是下一步——我要把这款产品推进日本的本地游戏平台。

这里有个现实问题：对日本平台来说，我们是一个完全陌生的外部产品，凭什么给我们过审，更别说给推荐资源位了。推荐位是稀缺资源，通常是给平台自己看好的产品的。

我的做法是，**不去谈关系，而是去谈数据**。我带着这款产品在 Facebook 上的评分和实际表现数据去跟平台谈，把“请给我们一个位置”这个请求，转化成“这是一款已经被验证过的、留存和口碑都达到什么水平的产品，把它放在推荐位对平台自己的用户价值是什么”。

最终产品通过了审核，也拿到了推荐资源位。

这个案例对我个人的意义是，我第一次验证了一套方法论：**用一个市场已经跑出来的数据，去说服另一个陌生市场给你资源**。这套思路我后来在 BlueStacks 说服美国总部投资资源的时候又用了一次，而且是更大的赌注。

三、平台型产品阶段

3.0 先讲清楚我对平台产品的理解（约 45 秒）

（被问“你怎么理解平台型产品”时用，也可以作为讲阿里/BlueStacks 之前的铺垫）

平台是十几年前在全球发展起来的一种产品形态，它的特点是自己不提供实质性的内容，而是搭建一个场子，让内容的供给方和消费方在上面做事——平台提供的是机制、流程和工具，帮助双方顺利完成各自想做的事。电商、游戏、影视、短视频、新闻，本质上都是这个逻辑。所以平台产品的难点在于，它同时要 2C 也要 2B。

我在这个阶段形成了一条判断原则，到今天还在用：**平台上的用户有一个核心诉求，其他功能可以尝试，但一旦某个功能脱离了这个核心诉求、变成“为了做而做”，它就跟产品的使命相违背，很难走通**。

这条原则不是我总结出来放在那的，后面我会讲到，我在 BlueStacks 用它砍掉过两个已经投入了资源的项目。

3.1 阿里：在陌生领域布局（约 45 秒）

（被问阿里经历时用，这段建议简讲，把时间留给 BlueStacks）

在阿里我负责的是天猫魔盒、淘宝手游这条线上的几块业务：主机云游戏的游戏引入和评估、开发者平台的数据分析模块、家庭互动玩法，也从零搭建过 HTML5 游戏平台，同时对接运营侧的需求。

这段经历对我最大的价值是，我近距离看到了一个大型互联网集团在进入一个自己不熟悉的新领域时，是怎么布局的、怎么投入资源、怎么允许试错的。

同时我也看到了反面——有些功能做出来，是因为组织内部觉得"应该有"，而不是因为用户真的需要。这让我形成了刚才说的那条原则，也让我做了一个职业上的决定：**之后我不再做这种脱离用户核心诉求的产品**。所以我下一步选择去一家真正深耕核心产品的公司。

3.2 BlueStacks 中国：从 0 到 1（约 90 秒）

（被问"从 0 到 1 的经历"时用）

我在 BlueStacks 待了八年。进去的时候的任务是从零搭建中国的产品和工程团队。

当时的核心判断有三个。第一，模拟器引擎是国际版积累下来的核心技术资产，这个必须保留，不能重造；但应用层要针对中国市场全部重新搭建，因为中国用户的使用场景和国际版完全不一样。第二，商业模式上，我引入了游戏收益分成的模式，而不是简单地卖广告。第三，我们需要一套后端运营平台来支撑这个模式跑起来，所以我搭建了包含游戏引入、数据分析、收益分成三个模块的游戏后端运营平台。

我在这里面起到的关键作用，是**明确产品形态、用户体验和盈利模式这三件事**，然后以此为基准去**反推构建每一个产品模块**。这些决策背后不是拍脑袋，是有行业调研、竞品分析和用户分析支撑的，也正是靠这些分析，我才能影响到美国总部的决策层。

结果是产品上线首月，日活就快速增长到十万。

3.3 BlueStacks：用数据说服美国团队做资源取舍（约 90 秒）

★ 这是我最推荐你作为"招牌案例"反复练的一段

（被问"一次重要的决策"、"怎么说服别人"、"数据驱动"、"跨部门冲突"时都可以用）

我讲一个当时压力最大的决策。

那个阶段我们面临一个资源取舍：引擎侧的研发资源，是投到开发一个新的技术方案上，还是集中投入到优化几款重点游戏的体验上——具体说就是性能、操作手感、引擎的自由配置这些。

这两个方向都有人支持，而且争论的方式是很典型的技术团队式争论：新方案更有想象空间，优化重点游戏更保守但更稳。这种争论本身是没有结论的，因为大家比的是对未来的直觉。

我做的事情是，把这个决策从"技术直觉之争"变成"商业判断"。我拿这几款重点游戏当时的评分数据，结合同类产品的历史表现数据，去**推测如果这些游戏的体验被优化之后，预期能带来多少用户量增长、能带来多少流水**。也就是说，我给"投入资源"这件事算了一笔预期回报的账。

有了这个测算，讨论的性质就变了。不再是"你觉得哪个更重要"，而是"这笔投入的预期回报是多少，我们认不认这个数"。

最终我说服了美国团队选择优化重点游戏这条路径，保证了线上产品的稳定性。

这次成功之后，我被任命为产品总监，开始负责全球产品线。

（如果被追问"如果数据错了呢"）我当时的测算是基于类比推测，不是精确预测，所以我在沟通的时候是把假设明确摆出来的——我说的是"如果这几款游戏的体验能达到同类产品的水平，对应的用户和流水大概在这个量级"。把假设讲清楚，对方就可以针对假设来挑战，而不是针对结论。这反而让讨论效率更高了。

3.4 BlueStacks 全球产品线（约 70 秒）

（被问"规模化"、"管理没有汇报关系的团队"时用）

接管全球产品线之后，产品峰值做到了千万级月活跃用户。商业化上我们拓展了多元的盈利模式，包括 CPI、CPS、游戏中心里的 banner 和推荐位，还有产品启动时的 splash screen 广告，收益持续增长，为公司带来了充足的现金流。

这个阶段最大的转变，是从"做成一件事"变成"建立一套可以被复制的体系"。

难点在于，我要跟重点运营国家的运营和市场团队对接，而这些团队跟我没有直接的汇报关系。他们有自己的 KPI，我不能靠指令推动他们。

我的解法是把游戏上线的全流程标准化和自动化，而不是每一次都靠人去协调。这个流程覆盖上线前的产品内外推广、预注册、CPI 和 CPS 的 SDK 接入、blog 制作、influencer 的视频和内容发布、游戏性能和操作的优化，一直到上线后的推广和竞品游戏的性能对比。

流程标准化之后，区域团队接到的不再是一堆临时请求，而是一套他们能预期、也能复用的东西——对他们自己的 KPI 也是有帮助的。这就把"我求你配合"变成了"我们共用一套工具"，协作成本大幅下降。

3.5 云游戏（约 75 秒）

（被问"技术+商业双重判断"、"成本控制"、"优先级判断"时用）

云游戏这条线是我在 BlueStacks 阶段技术和商业判断结合最紧的一段。

产品形态是：游戏在云端服务器上运行，通过 streaming 的方式把画面传输到用户终端展示。

我们前后遇到了四类挑战。

第一是 MVP 阶段的技术攻坚，要先证明这条路在技术上跑得通、体验能接受。

第二是上线后的盈利模式探索，我们试了启动广告和收益分成两条路。

第三是玩家量起来之后的 scaling 和成本控制。云游戏跟传统产品最不一样的地方是，它的成本是**随用户量线性增长**的——每一个在线玩家都在消耗一份服务器资源。所以用户增长在这里不是无条件的好事，单位经济模型必须先跑通。

第四，也是我认为最关键的一个判断，是游戏品类的取舍。重度大体量的游戏下载安装门槛很高，云游戏正好能解决这个痛点，这是最顺的场景。但同时也有像 Roblox 这类用户群体比较 niche 的产品，对它们来说，我们要算的是**服务器开销能不能跟收益打平**。

所以我在这条线上做的核心工作，其实不是把功能做出来，而是持续地判断：哪些品类值得承载，哪些不值得——这是一个必须同时懂技术成本结构和商业回报的判断。

3.6 Mod 和直播：两次主动止损（约 70 秒）

（被问“失败的项目”、“怎么做取舍”、“砍掉过什么”时用，这是一个加分项，不要回避）

我讲两个我主动砍掉的项目，因为我觉得这两个比成功案例更能说明我的判断逻辑。

第一个是游戏 Mod，就是让玩家自定义游戏内的资源——换皮肤、加特效、换场景元素。这个功能玩家是喜欢的，但我们始终没有找到可行的盈利模式，同时也没有得到游戏厂商的支持。原因很实际：大型游戏本身就有这个功能，而且是收费的，人家没理由让我们做；小游戏愿意配合，但它们在我们平台上的用户基数又不够，撑不起来。

第二个是游戏直播。我们让 influencer 在平台上一边玩游戏、一边用我们提供的 streaming 工具直播到海外的主流直播平台，同时也做了我们自己的直播模块给其他用户观看。做了一段时间之后我们判断，**这不是平台用户的核心诉求**——用户来我们平台是为了玩游戏，不是为了看别人玩。

这两个决定回头看，其实就是我在阿里学到的那条原则在起作用：脱离核心诉求的功能很难走通。区别在于，那条原则我不是挂在墙上，而是真的拿它去做“要不要继续投入”的判断，并且能在还有沉没成本的时候及时止损，不让项目死拖着不放手。

四、DemandTec：企业级 AI 产品

4.1 我现在负责什么（约 90 秒）

（被问“介绍一下你现在的工作”时用）

我 2023 年底加入 DemandTec，是一家美国总部的零售智能决策 SaaS 平台。加入的时间点比较特别，正好赶上生成式 AI 在企业场景落地的窗口期。

我现在的角色是 Director of Product Management，作为产品总负责人，全面负责 Data Engineering、Data Science、Platform 三大核心底层团队，支撑上层的业务应用层建设。

三个团队的分工是这样的：Data Science 团队负责 AI 能力建设，包含传统机器学习模型，也包含生成式 AI 和大模型的落地应用；Data Engineering 团队负责全链路的数据处理——数据接入、清洗、入库，以及预测结果的对外输出交付，我们面向的是大型零售商，要完成多门店、多分区维度的海量商品价格数据跟客户系统之间的双向导入导出；Platform 团队是中台，沉淀业务无关的通用平台能力，包括工作流 Farm、消息通知、搜索、信号处理、账号登录、权限体系这些公共模块，供上层所有业务应用复用。

需要说明的是，生成式 AI 和 Agent 在我们这里属于跨层解决方案，它同时覆盖底层能力和面向客户的交互层，所以这三个团队都参与相关研发。我因为早期就深耕这个方向，所以带团队承接了比较多的相关项目。

具体的产品管理职责上，我负责制定三大团队的整体产品策略和 Roadmap，做任务规划、资源分配和落地执行；横向跨团队对齐，并且深度对接重点客户，做定期的季度客户沟通，收集反馈、同步路线图和开发进度、管理客户预期；端到端负责核心模块从规划、内部交付到面向客户交付的完整闭环。

我目前管理 4 到 5 名直接下属，包括数据科学家、产品经理和建模服务相关人员。凭借业务产出，一年多的时间从技术产品经理晋升为产品总监。

4.2 Gen AI

从零到一搭建 Gen AI 的很多应用场景，

- 创建类：Pricing scenario creation，批量 offer 创建/API 导入
- 分析类：定价场景结果解读和分析，多定价场景 what if 分析，offer 历史分析，定价可解释性 - 通用 framework
- 数据和更新类：核心数据对话式查询和更新
- 提醒/通知/异常数据捕捉类：daily briefing 每日行动概览
-

4.2 ML

- offer 销量和利润预测模型
- 基础定价和促销预测模型中增加更丰富的因子：竞品价格变化，天气，库存
- 模型全流程自动化：从训练前的数据准备到在指定品类上运行模型，到 coefficients 输出，用于基于目标和约束的优化计算
- 模型效果评估，销量预测，促销预测效果评估和分析后台
- 购物篮分析，找出跨品类的光环效应产品，用于提供给客户 insights，以及提升模型预测准确性

4.3 模型监控与诊断评估体系（约 90 秒）

★ 这是你现阶段最推荐的招牌案例，最能体现 Hands-on + 客户价值

（被问“最有成就感的项目”、“hands-on 能力”、“怎么建立客户信任”时用）

我讲一个我觉得最有价值的项目：模型自动化监控与诊断评估体系。

先说问题。我们是给大型零售商做销量预测的，模型准确率直接关系到客户的定价和补货决策。但我们当时的状态是**被动的**——模型跑完，结果交付出去，如果有问题，通常是客户先发现、然后来报障，我们再回头去排查。这个过程对客户信任的伤害很大，因为每一次都是客户在替我们发现问题。

而且排查本身也很慢。预测是多维度的，出问题可能是某一类商品、某一个门店、某一个分区，靠人工去翻数据定位根因，周期很长。

我的做法是，自己动手用 **Web Coding** 快速搭了一个模型评估平台，没有走完整的立项排期。这个平台可以查看每一个客户、每一次预测任务的准确率和核心指标，并且支持多维度下钻分析，可以一层层定位到具体的商品、门店、分区，完成根因排查。

在这个基础上，我们实现了**主动的模型监控预警**——异常在客户发现之前就被我们自己发现。

结果有两层。第一层是量化的：基于这个平台的诊断能力，我们针对多个业务品类完成了模型调优，预测准确率提升了 3% 到 10% 以上。第二层，也是我更看重的：我们跟客户的关系从"出问题-被投诉-救火"变成了"我们主动告诉客户这里有异常、我们已经在处理"，客户对我们 AI 预测能力的信任度有了明显的提升。

（如果被追问"你自己写代码，跟工程团队边界会不会有冲突"）不会，因为定位很清楚。我搭的是原型，目的是快速验证"这个东西到底有没有用、需要看哪些指标"。这其实是我们公司在推的 **Sell before Build** 模式——先用原型验证市场和客户需求，再决定要不要投入正式研发资源。方向验证清楚之后，生产级的实现是交给工程团队做的。

4.3 促销预测新模型（约 50 秒）

（被问"最近在做什么"、"机器学习相关项目"时用）

我主导了一个新的促销预测模型的研发。

背景是，我们原有的能力主要覆盖常规销量预测，但零售商和供应商之间的促销业务是一个独立且复杂的场景——促销的销量和利润不是常规销量的简单变形，它受到促销机制、力度、周期等一系列因素影响。

我主导训练了一个基于决策树的促销 Offer 销量和利润预测模型，用的是两年的历史业务数据。目前已经完成了内部的 POC 验证，正在进入工程化落地阶段。

这个项目的产品意义在于，它拓展了我们对**零售商-供应商促销业务模式**的产品支撑能力，是从"我们能预测卖多少"到"我们能预测这次促销值不值得做"的一步。

4.4 Auto-DG 自动化需求组分派（约 60 秒）

（被问"算法/自动化项目"、"效率提升"时用）

Auto-DG 是我负责的另一个项目，全称是自动化需求组分派。

先解释一下背景。我们的销量预测是两阶段架构：先在需求组（Demand Group）这个层级做总量预测，再在组内做单品占比的拆分。这个架构的关键前提是，需求组要分得合理——如果把销售特征完全不同的商品放进同一个组，组级预测再准，拆到单品也是错的。

问题在于，这个分组过去是**人工完成的**。人工分组有两个问题：一是规模上不去，大型零售商的SKU 量级人工根本覆盖不过来；二是分组质量依赖个人经验，不一致也不可复现。

我把这个人工流程改造成了自动化流程，过程中完成了大量的算法优化工作。

结果是，在不同的业务维度下，模型的预测准确率普遍提升了 5% 以上。而且更重要的是，这个能力让我们在面对更大规模客户的时候，交付是可扩展的，不再受人工产能限制。

4.5 个性化推荐

1. 协同过滤（Collaborative Filtering）

最经典的一类，基于"物以类聚、人以群分"的假设。

- **基于用户/物品的邻域方法（User-CF / Item-CF）**：计算相似度做推荐，简单可解释，但冷启动和稀疏性问题明显。Item-CF 在工业界用得更多，因为物品相似度相对稳定，可以离线计算。
- **矩阵分解（Matrix Factorization，如 SVD、ALS）**：把用户-物品交互矩阵分解为低维隐向量，Netflix Prize 让这类方法出名。效果比邻域方法好，能处理稀疏数据，至今仍是很多召回环节的基线。

效果：在稠密数据、行为量大的场景下效果稳定，但对新用户/新物品几乎无能为力（冷启动），且难以融入内容特征。

2. 基于内容的推荐（Content-Based）

利用物品的属性特征（文本、类目、标签等）做匹配，常配合 TF-IDF、Embedding 相似度。

效果：能解决冷启动，可解释性强，但容易陷入"信息茧房"，推荐结果同质化，需要和协同过滤结合（混合推荐）。

3. 深度学习时代的主流模型

这是目前工业界的核心，通常分**召回**和**排序**两阶段：

召回阶段：

- **双塔模型（Two-Tower / DSSM）**：用户塔和物品塔分别编码成向量，用向量检索（ANN，如 Faiss）做海量候选召回。工业界（YouTube DNN 是代表作）广泛使用，效果好、延迟低。

排序阶段（精排，追求极致效果）：

- **Wide & Deep（Google）**：Wide 部分记忆（线性模型抓特征交叉），Deep 部分泛化（DNN 学习隐含模式），奠定了"记忆+泛化"结合的思路。
- **DeepFM / xDeepFM**：用 FM 自动学习特征二阶交叉，省去人工特征工程，效果通常优于 Wide & Deep。

- **DIN / DIEN（阿里）**：引入注意力机制，对用户历史行为按与目标物品的相关性动态加权，显著提升点击率预估准确度，是电商场景的代表性工作。
- **DCN / DCN-V2（特征交叉网络）**：显式建模高阶特征交叉，效果和效率的平衡做得比较好。

效果：这类模型相比传统 CF 通常能带来两位数百分比的 CTR/转化率提升（具体数字看场景），但训练和特征工程成本高，需要大量标注/行为数据支撑。

4. 图神经网络（GNN-based）

如 PinSAGE（Pinterest）、LightGCN，把用户-物品交互建模为图，通过邻居聚合传播信息。

效果：能捕捉更深层的关联（比如"看了 A 的人也看了 B，看了 B 的人又看了 C"），对长尾物品和冷启动有一定缓解，但计算复杂度和工程部署难度较高。

5. 序列/生成式推荐（近两年热点）

- **Transformer-based（SASRec、BERT4Rec）**：把用户行为序列当作"句子"，用自注意力建模行为的时序依赖。
- **大模型驱动推荐（LLM4Rec）**：用 LLM 做特征生成、冷启动内容理解，或直接生成式推荐（如生成候选物品 ID 序列）。目前还在快速迭代期，效果因场景而异，但在冷启动和跨域迁移上潜力较大。

综合来看的效果对比

方法	冷启动	可解释性	精度上限	工程成本
CF/矩阵分解	差	中	中	低
基于内容	好	高	低-中	低
Wide&Deep/DeepFM	中	低	高	中
DIN/DIEN(注意力)	中	低	很高	高
GNN	中-好	低	高	很高
序列/LLM	好	中	待验证/高	很高

工业界实践基本是**分层组合**：召回用双塔/CF 保证效率和覆盖率，粗排用简单模型快速筛选，精排用 DIN/DeepFM 类模型追求精度，再叠加规则做多样性/新颖性调控（避免过度个性化导致的信息茧房和疲劳）。

如果你是在评估某个具体场景（比如零售定价或者促销推荐里的个性化排序问题），我可以针对那个场景再展开讲讲适用的模型选型逻辑。

五、经历跨度大

(被问“你这几段经历跨度很大，游戏跟企业 SaaS 差别太远了吧”时用，不要辩解，正面承接)

这个问题我自己也想过很多次，我的答案是：表面上跨度很大，但底层能力是叠加的，不是每次重来。

工程阶段给我的是判断技术可行性的能力，这让我今天跟数据科学团队讨论模型的时候，不是只能听结论。

游戏阶段给我的是端到端交付一款完整产品的能力，以及用数据说服一个陌生市场给你资源的方法论——我在 6waves 说服日本平台、在 BlueStacks 说服美国总部，用的是同一套东西。

平台阶段给我的是两样：一是机制设计思维，不做单点功能，而是搭建让多方顺利完成诉求的流程和工具；二是判断力，知道什么时候该止损。

现在在 DemandTec，这几层是同时在用的——三个底层团队本质上就是中台平台，这是平台思维；用数据说服跨国团队做资源决策，这是前面练出来的；自己搭原型验证需求，这是工程底子。

所以我不认为我是从游戏“转”到了企业 SaaS，我是在一个更复杂的领域里，把之前所有的能力叠在一起用。

4

五、STAR 故事库（共 10 个，标注优先级）

同一个故事往往能回答 2-3 条不同 LP 的问题。二面建议优先准备故事 1 和 2，其余作为追问时的备选弹药。
✓ = 已准备好可直接用；■ = 需要你补充细节。

故事 1 | AI 分析 Ownership + Invent and Simplify

主打 LP: Ownership + Invent and Simplify, Customer Obsession / Dive Deep

我分享一个牵头用生成式 AI 解决跨产品线客户报告理解的项目。

背景是我对接客户和支持团队时发现一个系统性问题：客户看不懂系统生成的复杂数据报告，60% 的相关工单要升级到工程、客户成功甚至数据科学团队，单次解释周期 3-5 天，既拖慢客户决策，也消耗多团队人力，且会随客户增长持续恶化。

我的目标是做一套客户自助看懂报告的方案，降低人工介入率。基于经验，我判断生成式 AI 在复杂内容解释场景有明显优势，适合快速验证。

行动上我核心做了三件事：第一，小切口快速验证，先选了客户反馈最集中的 4 个高价值场景，用 vibecoding 的方式快速跑出第一版 demo，拉内部和客户评审和迭代。第一版出来后客户反馈“有的场景很有帮助，有的分析还不足以解决他们的问题，例如没答透价格变化的原因”，我没有当成简单的 prompt 优化问题，而是深挖客户真实疑问，最终发现根因：不是 prompt 不好，是解释引擎缺少商品间价格相关性的数据。补充相关性数据后，才真正解决了客户的核心疑问。第二，守住

风险底线。因为定价场景容错率极低，我坚持所有数字必须由确定性代码计算输出，大模型只负责组织语言，绝不允许生成或推断数值，同时设计了用户反馈闭环持续迭代。第三，做了架构层面的关键判断：把这个能力抽象成跨产品线复用的统一底层能力，统一输出格式和交互，保留场景化定制逻辑，避免各产品线重复造轮子。

结果方面，项目目前推进到 MVP 阶段，已交付 6 个灯塔客户试用。早期反馈积极：报告理解时间从几十分钟到几天，缩短到几分钟以内，还解锁了多维度定价情景对比这类过去没有的自助分析能力。因为还没正式全量上线，人工介入率的核心指标还没有最终数据，这也是我们上线后会重点追踪验证的。

Situation 过去两个季度，我在和客户、Support 团队沟通中发现一个跨产品线的系统性问题：客户拿到系统生成的复杂数据报告后经常看不懂，要联系 Support。Support 团队统计，大约 60% 的相关工单最终会升级到工程和客户成功团队，部分还要拉数据科学团队才能解释清楚，平均一次解释周期要 3 到 5 天。这不仅拖慢客户决策，也持续消耗三个团队的人力，而且随着客户量和功能增长，问题只会越来越严重。

Task 我设定的目标是：找到一个能让客户自己快速看懂报告的方案，把人工介入率降下来。基于经验判断，生成式 AI 在“复杂内容解释”这类场景上有明显优势，值得快速验证。

Action 第一步，我没有做大而全的方案，而是从客户反馈最集中、CS 介入率最高的场景切入——选了 4 个高价值场景（比如价格可解释性、定价情景对比），控制 demo 范围，用 vibe coding 快速跑通第一版，让内部团队和客户一起 review。

价格可解释性这个场景给了我一个很重要的信号：

第一版 demo 给 CS 和客户看完后，反馈是“确实有帮助，但还没能回答我们所有关于价格的问题”——比如客户经常问，做价格优化时，如果调整了选择的 UPC-location 范围，为什么同一个 UPC-location 算出来的价格结果会不一样。我没有把这当成“再调一下 prompt 就行”的小问题，而是带着团队重新回去和客户深入沟通，搞清楚他们真正想问的是什么。结果发现根源不是 prompt 不够好，而是支撑解释的数据本身不够丰富——价格优化本质上是把相关联的商品放在一起联合求解的，所以当参与优化的 UPC-location 范围变了，跟这个商品价格相关联的其他商品也跟着变了，联合求解出来的最优价格自然会变，但我们原来的解释引擎里没有这些商品间价格相关性的数据，AI 答不出这层因果关系。我们把价格相关性商品的数据补充进解释工作流之后，这一版才真正讲清楚“为什么价格会变”，覆盖了客户真正想问的问题。这件事也让我更确定一个原则：生成式 AI 能不能讲清楚，很大程度取决于喂给它的数据是否完整，不只是 prompt 写得好不好。

同时因为这类场景一旦讲错代价很大，我坚持了一个原则——所有数字必须由确定性代码算好，LLM 只负责组织语言，不允许它自己计算或推断数字，并设计了 human-in-the-loop 反馈闭环，客户可以标记“这条不准确”，反馈会被结构化地用来持续修正。

第二步，是架构上的决策，我认为这是整个项目里最关键的一个判断。我判断这不该是某个产品单独做的功能，而应该做成一个能被多条产品线复用的统一底层能力——统一输出格式和交互流程，但保留场景化的定制 prompt 逻辑，这样能避免每条产品线各自维护一套逻辑、重复造轮子。目前这个架构在 4 个场景下验证是可行的，但我把它当成一个持续验证的假设，而不是已经定型的最终方案——接入更多场景时如果发现现有抽象装不下，我会重新评估调整。

Result 项目目前从 Demo 推进到 MVP 阶段，已交付给 6 个灯塔客户试用——5 个 Pricing、1 个 Trade Intelligence。早期反馈是积极的：报告理解时间从原来的几十分钟甚至几天缩短到 1-2 分钟以内，同时有几个场景（比如多维度定价情景对比）是客户过去完全没有自助获取路径的分析能力，现在能直接从报告里拿到——这不只是把已有服务做快，也是解锁了新的洞察能力。项目还没有正式上线，所以 CS/ENG/DS 介入率下降这个核心假设还没有真实数据验证，这也是上线后我们要重点追踪的指标

数据准备具体怎么做的

说明哪些场景数据是现成的、哪些缺什么数据、你怎么判断优先补哪部分数据。

价格可解释性那次迭代（新加入的复杂时刻，重点准备）

案例已经具体化：客户问的是"调整 UPC-location 选择范围后，为什么同一个 UPC-location 的最优价格结果会变"，根因是价格优化对相关商品联合求解，解释引擎原来缺商品间价格相关性数据。面试官可能追问：

- 具体是怎么发现"没覆盖所有问题"的：是客户直接提的，还是你们观察到客户 demo 后还在联系 CS 问同样的问题？
- 深入访谈客户花了多长时间、和几个客户谈的
- "价格相关性商品的数据"具体是什么形式（比如商品间的关联规则、价格族/价格带定义），从哪个系统/团队拿到的
- 补充数据之后，怎么验证新版本真的解决了问题——有没有再给客户看一次，反馈如何
- 如果可以，准备一句真实的客户原话或反馈，比空泛讲"反馈很好"更有说服力
- 建议提前想好一句非技术背景也能听懂的两比，比如"这些商品的价格是绑在一起算的，换了一批商品做优化，跟它绑定的商品变了，算出来的最优价自然也会变"

幻觉/准确性机制（完整版，核心故事只带了"计算表达分离"一条）

- 输出后置的规则化校验：把生成结果里的数字和数据源做程序化比对比，不一致就拦截，不用"LLM 查 LLM"（后者本身也可能出错）
- 溯源标注：每个关键结论标明对应哪条基础数据
- 范围约束+越界检测：prompt 明确"只能基于提供的数据回答"，并检测输出是否引用了上下文之外的内容
- 分级展示：UI 上把"系统算出的硬事实"和"AI 生成的解读"视觉区分开
- 黄金测试集回归测试：每次调整 prompt 前先跑一遍已知场景，防止改动引入新问题
- 高风险场景保守阈值：可能直接影响客户定价决策的内容，置信度低于阈值时标"建议人工核实"
- 建议准备一个具体案例：某次客户标了"不准确"，原因是什么、怎么定位到是数据问题还是 prompt 问题、最后怎么改的——这比空讲机制更有说服力

反馈闭环具体怎么运作

- 标记时要求选具体原因类别：数字错误/结论误读/缺失上下文/表述不清晰/无关内容
- 明确 owner（你或对应场景的 DS/工程负责人），每周 review 一次
- 按严重程度分 SLA：数字类硬错误优先修复，解读类问题批量进入下一轮迭代
- 每次因反馈调整的 prompt 都要加入黄金测试集，防止未来回归
- 标记率(flag rate)按场景/时间的变化趋势，作为项目未上线阶段的早期质量代理指标
- 现实预期：现阶段 6 个客户的标记量还撑不起真正的模型微调，现实路径是持续 prompt 迭代

模型选型/评估框架具体怎么落地的

- 先用效果最强的 Claude Sonnet 跑通验证价值
- 建立黄金测试集，核心指标包括 grounding 率、幻觉率、任务相关性、一致性、延迟与成本
- 候选模型必须全部关键指标达标才能启用，不能只看成本更低就切换
- 按场景路由：简单场景可以用便宜模型，复杂推理场景继续用强模型，不是一刀切
- 离线评估通过后先小流量灰度，对比真实标记率，确认无退化再全量切换

客户数据敏感，怎么保证合规/隐私

- 数据最小化：只传分析必需的字段给模型 API
- 跨客户数据严格隔离，避免不同客户的数据在同一次调用/上下文里混用
- 考虑到总部在美国、你在国内办公，评估了跨境数据传输的合规要求

- 确认了供应商侧的数据处理协议（是否用于训练、保留周期）、内部对敏感反馈数据的访问权限管控

如何用 AI/vibe coding 加速的

我先想清楚统一架构该怎么设计，然后用 AI 辅助工具快速把 1 个方案原型跑出来验证，这样几小时内就能让团队看到实际效果，大家感到非常 promising 后又在 2 天时间完成了全部 4 个场景，可以开始组织 CS review 和 customer review。

故事 2 | 成式 AI 产品战略

LP: Ownership、Customer Obsession、Bias for Action、Invent and Simplify、Think Big、Deliver Results

精简口语版（约 90 秒，运营语言校准版，练到脱稿）

我讲一个我从 0 到 1 定义生成式 AI 产品方向的项目，最后的结果是，这个方向现在是公司 2026 年商业化路线图里最重要的一块，而且已经有 7 个标杆客户明确表达了付费意向。

背景是，2024 年公司决定投入生成式 AI，最初的打法是想做一个覆盖所有场景的大而全智能助手，投入了不少资源，但迟迟拿不出一个站得住脚的方案，项目实际上停滞了，后来负责这个方向的 VP 离职，一度没有人主导。

我复盘后判断，问题不在执行细节，而在于我们一直是“自己觉得该做什么”，没有真正拿着方案去客户那验证过。所以我提出把打法整个换掉：不再继续打磨一个大方案，而是快速在几个客户有切实痛点的场景上分别做出可演示的版本，一两天出一版，直接拿给客户看，用真实反馈说话，而不是内部拍脑袋定优先级。几个场景跑下来，客户反馈的迫切程度差异很明显，我们就用这个作为资源投入的依据，把最痛的场景优先落地。

这里有一个我坚持的运营原则：所有关键决策点必须留人工确认，系统负责把繁琐的操作自动化，但不替业务用户做最终判断——这直接关系到客户敢不敢把这个工具用在真实业务里。

结果是，目前有 7 个标杆客户明确表达了采纳意向，部分客户已经愿意为此付费。效率提升最明显的一个场景，原来客户要反复试错、跑几个小时才能完成的操作，现在不到 5 分钟就能做完。这个项目现在是公司 2026 年商业化路线图里最重要的一块。

（技术细节——Dify demo、Claude Code vs Dify 架构决策——留给追问时讲，不主动展开，避免又陷入技术讨论。下面是完整版 S/T/A/R 拆解，供深挖问题时用。）

Situation

2024 年，公司决定投入生成式 AI，我的 VP 最初想做一个覆盖全部业务场景的大而全 AI Copilot。团队推进了一段时间，始终设计不出一个站得住脚、让人信服的方案，整个团队都很困惑，项目实际上卡住了。不久后 VP 离职，公司在这个方向上一度没有人主导。

Task

新领导还没有对这个方向形成自己的判断，项目实际上处于停滞状态。我意识到不能让它继续悬着，需要先把问题的根源想清楚，再拿出一个可以快速验证的方向。

Action

第一步，诊断根因。 我们复盘发现，问题的根源不在执行细节，而在于我们几乎没有真正深入客户去找核心痛点，而是基于内部假设在设计方案——所以之前的方案既没有经过客户验证，也没有真实反馈可以迭代。这是一个方向性问题。

第二步，快速在多个候选场景上分别验证，而不是继续打磨一个大方案。 我提出放弃"一次做一个大而全 Copilot"，改成每个候选场景用 Dify 花一两天就搭出一版可演示的 demo，不追求生产级完整度，先让客户看到、给出真实反馈。首批 demo 出来之后，客户反馈非常不错。

第三步，用客户反馈定优先级，而不是内部拍脑袋。 有部分场景客户的反应明显更迫切，我们又跟更多客户沟通，把反馈汇总对比，用这个作为优先级判断的依据。

第四步，把优先级最高的场景真正落地。 确定方向后，我开始跟工程团队紧密协作，把初步 demo 进一步落实——接入真实数据、真实 API，把整个流程跑通。这个过程中我坚持一个原则：关键决策点必须人工确认，系统负责计算和数据处理，但不替业务用户做最终决策。

最初的 demo 是用 Dify 的可视化工作流快速搭出来的，但进入工程化、要做成可上生产的产品时，我判断不应该继续用 Dify 承载生产环境——可视化工作流每次改动都要回到 Dify 里手动调整，而且整个方案会长期依赖一个第三方框架，灵活性和可控性都受限。我提出改用 Claude Code 这类 AI 辅助编码的方式直接写代码来实现同样的工作流逻辑，既能达到一样的效果，又不受第三方框架限制、后续迭代也更灵活。这个判断最终被工程团队采纳，生产环境没有使用 Dify。

Result

目前已经有大约 7 个标杆客户表达了采纳意向，其中部分客户甚至愿意为此付费购买。各个场景带来的效率提升幅度不完全一样，但有一个例子特别能说明问题：在"添加约束条件（constraint）"这个场景里，原来的流程非常复杂，客户往往一次填不对，要反复试、反复跑一遍完整场景才能看到结果，整个过程要花几个小时；现在用 Agent 之后，同样的事情不到 5 分钟就能完成，而且不需要反复运行场景去试错。这个项目现在已经成为公司 2026 年商业化路线图里最重要的一块。这也是我个人第一次在没有人指派、团队一度群龙无首的情况下，独立判断方向、并快速把它验证出来的经历。

追问预案

1. "候选场景具体有哪些？最后哪个最迫切？" 四个候选场景：**auto attribution（自动归因）、pricing scenario 智能添加 constraints（约束条件）、核心数据查询、scenario comparison（场景对比）**。建议准备一句话说明每个场景大概解决什么问题，重点讲清楚"添加 constraints"这个是反馈最强烈、最终优先落地的场景——因为它原来的人工流程最痛（见 Result 里的具体数字）。
2. "7 个客户里有的愿意付费，具体是怎么谈的？定价逻辑是什么？" ■ 目前你确认的是"客户明确表示想购买这个 solution"，但具体定价机制（按场景/按用量/打包进现有合同）还没定。建议准备一句框架性的回答，比如"目前还在和销售/产品团队商讨具体的定价打包方式，但客户主动提出购买意向本身，是对这个方向价值最直接的验证"——不需要装作已经有了完整的定价体系，诚实说"还在细化"反而更可信。
3. "添加 constraint 这个场景，效率提升具体是怎么算的？" 已经写进 Result：原来一次不一定能填对，要反复尝试、反复跑完整场景看结果，整个过程要几个小时；现在不到 5 分钟，而且不需要反复运行场景去试错。这是目前最有冲击力的一个数字，建议这句话练到能脱口而出，语气自然、不要听起来像背数据。
4. "新领导来了之后，是怎么认可这个方向的？" 新领导看了 demo 就认可了，过程比较顺利。可以直接说"因为已经有首批客户的真实反馈作为证据，新领导看了 demo 很快就认可了这个方向"——不需要编造波折，顺利本身也说明了"用客户证据说话"这个方法论是有效的。
5. "跟工程团队'紧密协作'具体是怎么分工的？有没有冲突？" 分工很清楚：**你负责定义需求、验收标准、优化 prompt 和优先级；工程负责工程化实现，把 demo 做成可上生产的产品。** 冲突点在于生产环境要不要继续用 Dify 的可视化工作流——你判断不应该继续用，理由是：① 每次改动都要回到 Dify 里手动调整可视化流程，效率不高；② 长期依赖一个第三方框架，灵活性和可控性都受限。你提出改用 Claude Code 直接写代码实现同样的逻辑，同样有效但更灵活、不受第三方限制。工程团队最终采纳了这个建议，生产环境没有用 Dify。

故事 3 | 主动模型监控与诊断评估体系 ✓

主要 LP: Customer Obsession, Ownership, Dive Deep

Situation 我们给大型零售商做销量预测，模型准确率直接关系到客户的定价和补货决策。但当时我们的状态是被动的——模型跑完、结果交付出去，如果哪里出了问题，通常是客户先发现、然后来报障，我们再回头排查。这个过程对客户信任的伤害很大，因为每一次都是客户在替我们发现问题。而且排查本身也很慢：预测是多维度的，出问题可能是某一类商品、某一个门店、某一个分区，靠人工翻数据定位根因，周期很长。

Task 我给自己定的目标是，找到一个能让我们在客户之前发现问题的办法，把这个被动响应的模式变成主动发现、主动预警。

Action 我没有走完整的立项排期，自己动手用 Web Coding 快速搭了一个模型评估平台原型——这其实是我们公司在推的 Sell before Build 模式，先用原型验证需求和价值，再决定要不要投入正式研发资源。这个平台可以查看每一个客户、每一次预测任务的准确率和核心指标，并且支持多维度下钻分析，能一层层定位到具体的商品、门店、分区，完成根因排查。在这个基础上，我们实现了主动的模型监控预警——异常在客户发现之前就被我们自己发现。

Result 结果有两层。第一层是量化的：基于这个平台的诊断能力，我们针对多个业务品类完成了模型调优，预测准确率提升了 3% 到 10% 以上。第二层，也是我更看重的：我们跟客户的关系从“出问题-被投诉-救火”变成了“我们主动告诉客户这里有异常、我们已经在处理”，客户对我们 AI 预测能力的信任度有了明显的提升。

追问预案

· 你自己写代码，跟工程团队边界会不会有冲突？ 不会，因为定位很清楚。我搭的是原型，目的是快速验证“这个东西到底有没有用、需要看哪些指标”。方向验证清楚之后，生产级的实现是交给工程团队做的。

· 这个和故事 4（模型评估框架）是不是同一个项目？ 不是同一个。这个故事讲的是从被动响应到主动监控的转变，核心是客户信任的建立；故事 4 讲的是需求组自动分派和全链路自动化，核心是标准化和可扩展性。两个都涉及模型评估体系，但解决的问题和体现的 LP 不一样，可以按面试官问的角度选用。

故事 4 | Auto-DG 自动化需求组分派与模型全链路自动化 ✓

主要 LP: Dive Deep, Insist on the Highest Standards, Deliver Results

Situation 我们的销量预测采用两阶段架构——先在需求组（Demand Group）这个层级做总量预测，再在组内做单品占比拆分。这个架构有一个关键前提：需求组要分得合理，如果把销售特征完全不同的商品放进同一组，组级预测再准，拆到单品也是错的。问题是，这个分组过去完全靠人工完成，有两个硬伤：一是规模上不去，大型零售商的 SKU 量级人工根本覆盖不过来；二是分组质量依赖个人经验，不一致也不可复现。

Task 我给自己定的目标，是把这个人工流程改造成自动化流程，同时保证甚至提升分组质量，让交付能力不再受人工产能限制。

Action 我主导了这个从人工到自动化的迁移（Auto-DG），过程中做了大量算法优化工作——不是简单地把人工规则写成代码，而是重新设计了分组逻辑本身，让它能基于商品的销售特征数据自动完成合理分组。同时，我们也在這條产品线上持续投入模型全流程自动化建设：从训练前的数据准备，到在指定品类上运行模型、输出用于优化计算的系数，再到销量和促销预测的效果评估分析后台，我建立了一套标准化的评估框架和实时监控 dashboard，还引入了购物篮分析，找出跨品类的光环效应商品，既能给客户id提供额外洞察，也反过来提升了模型的预测准确性。

Result 结果是，在不同的业务维度下，模型的预测准确率普遍提升了 5% 以上；核心场景的模型运行效率提升了 40%，预测的 MAPE 降低了 5 到 10 个百分点。而且更重要的是一个可扩展性的结果：这套自动化能力让我们在面对更大规模客户的时候，交付是可扩展的，不再受人工产能限制。

追问预案

· 自动分组的算法逻辑大致是什么？ 核心是基于商品的历史销售特征（比如销量曲线的相似度、价格弹性等维度）做聚类，再结合业务规则做校正，避免把业务逻辑上不该在一起的商品分到同一组。

· 新业务线从零开始建模型，最大的难点是什么？ 最大的难点是没有历史模式可以参照，我们需要先用有限的数据验证一个可行的基线模型，再逐步补充特征和数据，这个过程用的评估框架和监控 dashboard 跟故事 3 是一套体系的延伸。

故事 5 | BlueStacks 中国 0-to-1 落地与全球化规模复制 ✓

主要 LP: Ownership, Bias for Action, Think Big, Hire and Develop the Best

Situation BlueStacks 是一家美国总部的安卓模拟器公司，在中国市场当时完全没有本地化产品和团队。我加入时的任务，是从零搭建中国的产品和工程团队，把这个市场从头做起来。

Task 我需要在几乎没有现成参照的情况下，同时决定产品形态、盈利模式、以及支撑这一切的后端能力，并快速把它落地上线；站稳之后，还要把这套打法规模化复制到全球其他国家。

Action 我当时做了三个核心判断。第一，模拟器引擎是国际版积累下来的核心技术资产，必须保留，不能重造；但应用层要针对中国用户的使用场景全部重新搭建，因为跟国际版完全不一样。第二，商业模式上，我引入了游戏收益分成模式，而不是简单地卖广告。第三，我们需要一套后端运营平台来支撑这个模式跑起来，所以我搭建了包含游戏引入、数据分析、收益分成三个模块的游戏后端运营平台——先明确产品形态、用户体验和盈利模式这三件事，再以此为基准反推构建每一个产品模块，这些判断背后有行业调研、竞品分析和用户分析支撑，也正是靠这些分析，我才能影响到美国总部的决策层拿到资源支持。中国上线站稳之后，我接管了全球产品线，规模化上最大的挑战是，我要跟重点运营国家的运营和市场团队对接，而这些团队跟我没有直接汇报关系，他们有自己的 KPI，我不能靠指令推动。我的解法是把游戏上线的全流程标准化和自动化——覆盖上线前的产品内外推广、预注册、CPI 和 CPS 的 SDK 接入、blog 制作、influencer 内容发布、游戏性能优化，一直到上线后的推广和竞品对比。流程标准化之后，区域团队接到的不再是一堆临时请求，而是一套他们能预期、也能复用、对他们自己 KPI 也有帮助的东西——这就把“我求你配合”变成了“我们共用一套工具”，协作成本大幅下降。

Result 产品上线首月，中国区日活就快速增长到十万；接管全球产品线之后，峰值做到千万级月活跃用户，商业化上拓展了 CPI、CPS、banner 推荐位、启动广告等多元盈利模式，为公司带来了持续增长的现金流。这次成功也让我被任命为产品总监，开始负责全球产品线。

故事 6 | 用数据说服美国团队做资源取舍 ✓

主要 LP: Earn Trust, Have Backbone and Disagree and Commit

Situation 在 BlueStacks，我们面临一个资源分配的分歧：工程资源该投到开发一个新技术方案上，还是集中投入优化几款现有的重点产品体验（性能、操作手感、引擎的自由配置）。两派工程师都有支持者，争论的方式很典型——一方看重想象空间，一方看重稳妥——这种争论没有结论，因为比的是对未来的直觉，而我作为产品负责人，没有直接的决策权去拍板，只能靠说服来推动方向。

Task 我需要在没有直接汇报关系、且双方都有合理立场的情况下，推动团队对齐到一个方向，同时保证这个决定不是靠“谁声音大”，而是有据可依。

Action 我做的事情，是把这场“技术直觉之争”变成一次“投资回报测算”。我拿这几款重点产品当时的用户评分数据，结合同类产品的历史表现，推算如果把体验优化到位，预计能带来多少用户增长、多少收入——也就是给“投入资源”这件事算了一笔预期回报的账。有了这个测算，讨论的性质就变了：不再是“你觉得哪个更重要”，而是“这笔投入的预期回报是多少，大家认不认这个数”。当时有人追问“如果数据测算错了怎么办”，我的应对是把假设摆明——我说的是“如果这几款产品的体验能做到同类水平，对应的用户和收入大概在这个量级”，这样对方可以针对假设去挑战，而不是针对结论吵，讨论效率反而更高了。最终美国团队选择了优化现有重点产品这条更稳妥的路径。

Result 这个决定保住了线上产品的稳定性，也避免了资源分散在一个还没验证的新方向上。这次之后，我被任命为产品总监，开始负责全球产品线——这次经历也让我确认了一条方法论：遇到分歧，不是比谁更坚持，而是把直觉之争转化成可验证、可讨论的商业判断，这条原则我后来在 BlueStacks 全球化和 DemandTec 一直在用。

追问预案

· 如果对方还是不认同你的数据呢？ 我会把假设摆得更清楚，邀请对方一起挑战假设而不是结论；如果沟通后仍有分歧，我会清楚说明我的立场和风险，必要时升级给更高层做最终判断，但一旦团队决定了方向，我会全力支持执行，不会消极配合。

· 这算不算真正的 disagree and commit？你后来完全认同这个方向吗？ 算。测算本身就是我说服自己的过程，不是先有结论再找数据；这次经历也让我确认了这套方法论在跨团队没有直接汇报关系时依然有效，后来在

BlueStacks 全球化和 DemandTec，我一直在用同样的逻辑推动对齐。

故事 7 | 高并发平台稳定性优化（零增量成本）✔

主要 LP: Dive Deep, Insist on the Highest Standards, Ownership

Situation 随着客户数量和数据量增长，平台在高并发场景下出现了明显瓶颈——多个大客户的批量计算任务挤在同一时间段跑，原本预期几个小时能出结果的任务群组，实际要跑 1 到 2 天才能完成。这不只是体验问题，背后还有一个商业压力：常规的解法是让客户加钱买更多服务器资源，但我们并不确定问题是不是真的出在资源不够，还是软件层面本身效率有问题。

Task 我给自己定的目标是，在不增加服务器成本的前提下，把这批高并发任务的完成时间大幅压缩，同时保证真正紧急的任务能被优先处理，而不是所有任务挤在一起排队。

Action 我做了三件事。第一，模型端优化：我们发现模型运算流程里有不少不必要的计算步骤，砍掉之后单次模型运行时间缩短了 90%；同时又优化了应用层的调用逻辑，在此基础上再缩短了 30%——相当于让每一个任务本身跑得更快得多。第二，资源利用率优化：在服务器资源完全不变的前提下，重新调整了并发任务数的配置，让同时跑的任务能更充分地占满现有算力，而不是资源闲置或者过载。第三，优先级调度：我重构了任务调度系统的优先级机制，把“客户在等结果、需要立即响应”的高优先级任务，和“可以等、不紧急”的低优先级任务区分开——高优先级任务优先跑，低优先级任务先排队，等系统空闲时段再执行，不再跟紧急任务抢资源。

Result 原来高并发时需要跑 1 到 2 天的任务群组，优化后 2 到 4 小时内就能完成，而且高优先级任务能被保证优先安排。客户对这个结果非常满意，因为问题彻底解决了，而且完全不需要他们额外付费增加服务器资源——这也证明了一件事：平台稳定性问题很多时候不是靠砸钱堆资源解决的，而是靠架构和调度效率。

追问预案

- 为什么不直接建议客户加资源，这样不是更快吗？加资源是最直接但成本最高的方案，我判断在没有真正验证过软件层面还有多少优化空间之前，不应该让客户为本可以避免的成本买单，所以先做效率优化，把加资源当成最后手段，不是第一选择。
- 高优先级任务会不会一直插队，导致低优先级任务永远排不上？低优先级任务会在系统空闲时段自动执行，不是无限期搁置，我们对排队等待时间也设了上限，避免真的被“饿死”。

故事 8 | Funcom 跨国团队协作：用预审机制减少返工 ✔

主要 LP: Earn Trust, Deliver Results, Learn and Be Curious

Situation 在 Funcom，我是项目经理兼美术总监，参与 The Secret World、Age of Conan 这些项目，管理三十多名美术师，团队分布在奥斯陆和中国两地。一个很典型的场景是，奥斯陆的动画总监对一个 monster 的动画有非常具体的要求，但中国团队理解不到位，做出来的东西反复达不到预期，来回返工了好几轮。

Task 我需要在保证交付质量和时间的前提下，解决这种跨文化、跨语言的理解偏差问题，而不只是充当翻译。

Action 我做了两件事。第一，我直接介入双方的讨论，不只是做语言上的翻译，而是帮双方对齐理解——总监想要的效果到底是什么、中国团队觉得难在哪里，然后在“需求标准”和“实现难度”之间找一个双方都能接受的折中方案。第二件事，也是更关键的一件，是我在团队把作品提交给总监做正式 review 之前，先按我自己对需求的理解做一轮预审——因为我全程参与了需求讨论，我大致知道总监会在哪些点上卡住，所以能提前把偏差找出来、让团队先改一轮，而不是等正式 review 时才发现问题。

Result 返工次数明显下降，整体交付效率提升，多个 AAA 级项目都按时交付。而且这个机制建立起来之后，总监对我的判断产生了信任，后续类似的对齐工作更多直接交给我处理。这种跨区域协作的能力后来在 BlueStacks（美国/印度）和 DemandTec（美国总部）反复被验证有效——我更擅长的不是事后救火，而是提前设计一个机制把问题挡在前面。

故事 9 | Hands-on AI Builder: Signal（副业项目，对应 JD 的 vibe-coding 要求）



主要 LP: Invent and Simplify, Learn and Be Curious, Ownership, Dive Deep

Situation 找工作这件事本身很重复、很耗时——刷职位、判断匹配度、针对每个职位调整简历、跟踪投递进度，这些工作琐碎但又不能跳过。

Task 作为一个个人项目，我给自己定的目标是独立设计并搭建一个端到端的 AI 求职助手，从问题定义到做出一个接近生产质量的工具，全流程一个人完成。

Action 我搭建了一个多模型抽象层，同时接入 Claude 和 DeepSeek，支撑 6 个以上不同的 AI 工作流，按任务类型做模型路由，在成本和效果之间做平衡——不是所有任务都要用最贵的模型。我还搭了一套稳健的异步处理管道（Flask + SQLite WAL + 后台线程），用来处理耗时较长的大模型调用，避免阻塞。在浏览器自动化这块，我用 Playwright 实现了职位申请的自动填表，但刻意设计成“只自动填、绝不自动提交”——这是一个写进架构里的人工确认边界，不是靠一句 prompt 去约束模型。

Result 项目最终有一万行以上代码，用 8 个自动化测试套件做了完整的回归测试（LLM 调用做了 mock，不依赖真实 API），并且在 GitHub 上开源了。

追问预案

· **为什么要坚持自动填表但不自动提交？** 我判断求职申请是一个高风险、不可逆的动作——一旦提交出错的信息，没有撤回的机会，所以人工确认这一步不能省。这跟我在故事 1 里坚持“数字必须由确定性代码算、AI 不能自己下结论”是同一套思路：AI 负责提效，人负责兜底判断。

· **为什么要接入两个不同的大模型？** 主要是成本和效果的平衡。简单的任务（比如格式化、字段抽取）用更便宜的模型就够了，复杂的推理任务（比如匹配度打分、简历定制）用效果更强的模型，按任务路由而不是一刀切，这跟我在故事 1 里模型选型的逻辑是一致的。

故事 10 | Jira 自动化：PES → DMAN Epic（备选）■ 需补充量化结果

主要 LP: Invent and Simplify, Deliver Results

Situation PES（DTPES）团队的 Objective 工单，需要手动转换成 DMAN 团队对应的 Epic——字段同步、组件映射、issue linking 全靠人工完成，既耗时又容易出错，尤其是团队规模扩大之后，这类跨工具的工单转换成了一个持续消耗人力的重复性工作。

Task 我给自己定的目标，是提升这个跨工具工单转换流程的效率和一致性，把人工容易出错的环节自动化掉。

Action 我通过 Rovo Skills 构建了一套自动化流程：从 PES Objective 自动创建结构化的 DMAN Epic，自动同步 story points、优先级、起止日期、fix version 等关键字段，做组件映射，并建立“relates to”关联，让两个团队的工单始终保持对齐，不需要人工来回核对。

Result ■ 待补充：这里需要一个具体数字才有说服力，比如——每次转换原来大概要花多少时间、自动化后压缩到多少；人工转换时容易出错的环节（比如漏填字段、关联错误）大概发生率是多少、自动化后有没有降到接近零；目前累计覆盖了多少个 Epic。哪怕是一个大致的估算范围也可以，比如“原来每个工单大概要花 20-30 分钟，现在几乎是实时生成”。

待办：如果面试时间充裕、需要第三个故事，可以作为备选，但建议先把 Result 的量化数字补上再使用——没有数字的 STAR 故事说服力会明显打折。

六、技术问题

除了故事 1 自带的两个追问预案（幻觉处理、成本质量权衡），以下是其他可能出现的技术/AI 深挖问题：

可能的问题	建议怎么回答（对应你的经历）
你怎么评估一个 AI agent / LLM 功能上线前是否 ready?	故事 4 的评估框架 + 监控 dashboard，以及 Signal 项目里 8 个自动化测试套件（mock LLM 调用）的做法
描述一下你设计 agent 工具编排 / human-in-the-loop 的思路	故事 2 的 workflow & tool orchestration 设计 + 故事 9 的 auto-fill-never-auto-submit 安全边界
讲一个你需要把业务需求转成技术需求、并和工程对齐的例子	故事 2 或故事 4
你怎么看待 AI 产品里的 responsible AI / 数据隐私?	故事 1 的 human-in-the-loop 反馈闭环 + 分层防御设计

七、产品判断力 / MVP 优先级问题

常见问法：“如果要给这个 AI Agent 产品定义一个 MVP，你会优先做什么？为什么？”建议提前想一个具体切入点，可以直接引用第五节里的框架，例如优先做“合规材料智能预审”或“流失预警”，并说明选择依据（影响面大、技术可行性高、可快速验证价值）。

另一个常见问法：“你怎么衡量一个 AI 产品的成功？会用什么指标？”建议准备：采用率、任务完成时间缩短比例、人工介入率下降、客户满意度（NPS）等，并结合故事 1 里“报告理解时间从几十分钟缩短到一两分钟”这个真实数字。

统一作答框架（避免想到哪说到哪）：① 明确用户与场景 → ② 痛点与假设 → ③ 成功指标（北极星+护栏）→ ④ MVP 范围与取舍 → ⑤ 风险与应对 → ⑥ 上线策略与反馈闭环。

八、常见问题清单

离职原因（Why are you leaving）

中文版：

目前公司正在做一个战略调整，把产品团队相关职能转移到海外，以便更贴近海外客户、提升响应效率。这个变化让我也重新思考了自己下一步的职业方向——我更希望去寻找一个重点客户群体在中国的国际化产品，我希望在这样的环境里深耕，且能持续做 AI 方向的公司。这也是我特别看中亚马逊这个职位的原因。

English:

My current company is going through a strategic shift, moving product teams and related functions overseas to better serve our international customers. This transition prompted me to think more intentionally about my own next step — rather than waiting through that transition, I wanted to proactively find a platform where I could keep growing in a global product environment, especially one with a strong focus on AI. That's what led me to start exploring opportunities at international companies.

Why Amazon / Why this role

中文版：

2. 亚马逊的技术，平台和实力：我一直很欣赏亚马逊在技术深度和客户导向上的坚持——Customer Obsession 不只是一句话，而是真正体现在产品决策的每一个细节里，这跟我自己做产品的方法论是高度契合的。

3. 核心业务和客户群：目前公司正在做一个战略调整，把产品团队相关职能转移到海外，以便更贴近海外客户、提升响应效率。这个变化让我也重新思考了自己下一步的职业方向——我更希望去寻找一个重点客户群体在中国的国际化产品，我希望在这样的环境里深耕，这个职位完美满足了这个期待。

4. AI (gen+ML/DL) 我这两年在生成式 AI 方向投入了大量精力，不管是在实际工作中设计 agentic workflow，还是自己动手做原型验证，我都希望接下来的职业发展能持续深耕这个方向。而这个 NBS AI PM 岗位，正好把'AI 技术落地'和'跨境电商这种复杂的、多方协作的业务场景'结合在一起，这跟我过去在 DemandTec 做定价和促销 AI Agent 的经验是高度相关的，也是我觉得自己能快速上手并创造价值的地方。

English:

I've always admired Amazon's depth in technology and its genuine customer obsession — not just as a principle, but something that shows up in real product decisions. That resonates strongly with how I approach product work myself. I've also spent the past couple of years deeply focused on generative AI — designing agentic workflows in production and building prototypes hands-on — and I want my next role to let me keep growing in that direction. This NBS AI PM role combines exactly that: applying AI in a complex, cross-functional business like cross-border e-commerce, which closely mirrors what I did at DemandTec building AI agents for pricing and promotions. I think that's where I can add value quickly.

身份与职业规划类（大概率已在一面覆盖，二面无需主动展开）

- 请做一个简短的自我介绍 / Walk me through your resume（见 4.1，最高优先级）
- 你期望的薪资范围是多少？
- 最早什么时候可以到岗？是否需要考虑当前公司的通知期？
- 你未来 3-5 年的职业规划是什么？（结合“深耕 AI Agent 产品方向”来回答，逻辑要与 Why Amazon 呼应）
- 你还在面试哪些公司/岗位？（如实但简洁回答，不必展开细节）

待办：薪资范围和到岗时间需要你自己定一个数字/范围——建议提前想清楚，并了解一下上海市场 Senior PM/Director 级别的行情作为参考。到岗时间要确认当前公司的通知期要求。

行为面试类（Amazon LP 高频）

- 讲一个你曾经犯错/失败的经历，你从中学到了什么（LP: Learn and Be Curious / Deliver Results）
- 讲一个你和上级/同事意见不合的经历，你是怎么处理的（LP: Have Backbone; Disagree and Commit）→ 故事 6
- 讲一个你需要在多个优先级之间做取舍的经历（LP: Deliver Results / Bias for Action）
- 讲一个你没有足够资源/授权、但依然推动项目完成的经历（LP: Ownership / Invent and Simplify）→ 故事 2 / 9
- 讲一个你说服跨部门团队接受你的想法的经历（LP: Earn Trust）→ 故事 1 / 6
- 讲一个你收到负面反馈后如何应对的经历（LP: Earn Trust / Learn and Be Curious）
- 你如何定义“好的产品经理”？举一个你认为自己做得好的例子

业务理解与技术类

- 你怎么理解 NBS/跨境电商这个业务？→ 见第五节，已充分准备
- 如果要给这个 AI Agent 产品定义一个 MVP，你会优先做什么？为什么？→ 见第八节
- 你怎么衡量一个 AI 产品的成功？会用什么指标？→ 见第八节
- 如果数据科学团队和业务方对优先级有分歧，你会怎么协调？→ 可结合故事 6 的框架

九、简历中可能被追问的薄弱点，及应对策略

潜在薄弱点	应对思路
没有直接的 Amazon / 电商平台卖家端经验	强调 12+ 年跨境 B2B/B2C 经验；定价/促销/需求预测决策平台与卖家经营效益高度相关；BlueStacks 全球平台从 0 到 1000 万 MAU 的规模化经验
之前公司规模比 Amazon 小	强调 0-to-1 owning 的广度（同时负责 Data Science / Platform / Data Engineering 三条线），恰恰证明能在更大平台上做得更好
“全球贸易合规”是 preferred，不是你的强项	坦诚这不是过去的核心领域，主动衔接到正在带的 compliance review workstream，以及数据隐私/账户安全（MFA）上的实际经验
为什么在 DemandTec 刚升职（2025 年 10 月）就考虑离开？	见第四节 4.2，聚焦公司战略调整这一客观因素，不要负面评价现东家

十、可以反问面试官的问题

- 这个 AI 产品线目前处于什么阶段（0-1 还是规模化）？
- 这个团队与 Central Product / MP team 的分工边界是怎样的？CN 和 MP（海外 marketplace）之间日常的协作节奏是怎样的？
- 这个 NBS AI 项目和 Amazon 最近对外发布的 Seller Assistant（基于 Bedrock/Nova/Claude 的 agentic AI）是什么关系——是互补还是会逐步融合？
- 当前这个岗位/团队面临的最大挑战是什么？
- 这个角色成功的第一个 90 天大概是什么样子？
- （对 Bar Raiser 可问）在这个团队/这个岗位上，什么样的人容易做得很好，什么样的人容易遇到困难？

以下是二面专门准备的、针对 Hiring Manager 的反问问题：

- （针对 Hiring Manager）你目前怎么衡量这个岗位的成功？入职后的前 90 天，你希望看到什么？
- （针对 Hiring Manager）你觉得这个团队在运营 / AI 落地上，目前最大的痛点或瓶颈是什么？
- （针对 Hiring Manager）这个岗位平时和你的协作方式是怎样的？多久会有一次一对一？
- （针对 Hiring Manager）团队目前的组织结构是怎样的——有多少 PM、你直接管理哪些职能、和其他团队的分工边界在哪？

十一、面试当天技巧与 Checklist

- 30 分钟很紧，注意控制每个故事的时长（S+T 共 30 秒，A 45-60 秒，R 20 秒），不要一次性讲完所有细节，留空间给追问
- 严格用 STAR 结构，Result 部分一定要有可量化的数字
- 讲故事时多用 “I” 而不是 “we”，清楚说明你个人的决策和贡献
- 准备好被追问“如果重来一次会怎么做不同”或“团队里有人不同意你怎么办”
- 每个故事能挂上具体的 LP，但讲的时候不要说“这体现了 Ownership”，让面试官自己判断
- 每一轮准备 2-3 个问题反问面试官
- 面试结束后可以发一封简短的感谢邮件，重申你对这个角色和团队使命的兴趣

附录：LP × 故事速查表

Leadership Principle	对应故事
Customer Obsession	1, 3
Ownership	2, 3, 5, 7, 9
Invent and Simplify	1, 2, 9, 10
Think Big	2, 5
Dive Deep	3, 4, 7, 9
Insist on the Highest Standards	4, 7
Bias for Action	5
Hire and Develop the Best	5
Earn Trust	6, 8
Have Backbone; Disagree and Commit	6
Deliver Results	2, 4, 8, 10
Learn and Be Curious	8, 9